

FECAP – Fundação Escola de Comércio Álvares Penteado Curso de Ciência da
Computação

**SISTEMA IOT PARA MONITORAMENTO CONTÍNUO DA CADEIA DE FRIO DE
VACINAS FlexHealth**

Entrega 02 – Implementação e Validação do Sistema

Nomes dos integrantes:

Ettore Grecco (RA: 23025294)

Rafaella Morelli (RA: 23024902)

Luis Felipe Sparrapan (RA: 23025521)

Rodrigo Gama (RA: 23025472)

1. Objetivo do sistema

O projeto FlexHealth tem como objetivo desenvolver uma plataforma IoT inteligente capaz de monitorar continuamente as condições ambientais (temperatura e umidade) em refrigeradores e caixas térmicas destinados ao armazenamento de vacinas.

A solução visa garantir a integridade da cadeia de frio, assegurando que os imunizantes permaneçam na faixa de temperatura estritamente segura, definida entre 2°C e 8°C. O sistema atua de forma preventiva e corretiva, oferecendo:

- Monitoramento em tempo real via dashboard web;
- Detecção imediata de anomalias e quebra de limites térmicos;
- Alertas visuais locais (Display OLED);
- Histórico de leituras para auditoria e controle de qualidade;
- Transmissão segura de dados via protocolo MQTT com criptografia TLS.

2. Descrição da arquitetura final implementada

2.1 Visão geral da arquitetura

A arquitetura final implementada segue um fluxo linear e seguro de captura, transmissão e exibição de dados. O microcontrolador ESP32 é responsável pela coleta das leituras do sensor DHT22 e pelo acionamento da interface local (Display OLED).

Os dados são transmitidos via Wi-Fi utilizando o protocolo MQTT (encapsulado em TLS/SSL) para um broker em nuvem (HiveMQ). Um backend desenvolvido em Python atua como *subscriber*, recebendo, validando e persistindo esses dados em um banco de dados relacional SQLite. Por fim, uma API REST (Flask) serve essas informações para um Frontend reativo desenvolvido em React, que exibe os dados atualizados aos administradores do sistema.

2.2 Componentes da arquitetura

2.2.1 Sensores O sistema utiliza um sensor digital conectado ao ESP32 para capturar as variáveis ambientais críticas do refrigerador.

- **DHT22:** Responsável pela leitura simultânea de temperatura (precisão de $\pm 0.5^{\circ}\text{C}$) e umidade relativa do ar. As leituras são realizadas continuamente no loop principal do microcontrolador a cada 2 segundos.

2.2.2 Módulo de processamento — ESP32 O ESP32 foi programado em C++ utilizando a IDE do Arduino e executa as principais tarefas embarcadas:

- Conexão com a rede Wi-Fi local;

- Estabelecimento de túnel seguro (WiFiClientSecure) com o broker MQTT HiveMQ Cloud na porta 8883;
- Leitura dos dados do sensor DHT22;
- Processamento da lógica de negócio (validação da faixa segura de 2°C a 8°C);
- Atualização do Display LCD/OLED local;
- Montagem e publicação do payload JSON contendo ID do dispositivo, temperatura, umidade e status.

O código implementa uma máquina de estados binária para o status ambiental:

- **NORMAL:** Temperatura entre 2.0°C e 8.0°C.
- **ALERTA:** Temperatura inferior a 2.0°C ou superior a 8.0°C.

2.2.3 Atuadores e Interface Local Para garantir que o operador do laboratório tenha visibilidade imediata sem depender do acesso ao site, o sistema conta com respostas físicas:

- **Display LCD nativo do ESP-32:** Exibe em tempo real os valores de temperatura, umidade e mensagens de status (Ex: "STATUS: NORMAL" ou "ALERTA CRITICO").
- **2.2.4 Comunicação MQTT** A comunicação entre o ESP32 e o backend ocorre utilizando o broker HiveMQ Cloud.
- **Tópico de publicação:** flexhealth/geladeira01/dados
- **Exemplo de Payload publicado:**

JSON:

```
{
  "dispositivo": "geladeira_01",
  "temperatura": 7.6,
  "umidade": 50.0,
  "status": "NORMAL"
}
```

2.2.5 Backend e API O backend foi desenvolvido em Python, dividido em dois serviços simultâneos:

- **Receptor MQTT (backend.py):** Conecta-se ao broker usando a biblioteca paho-mqtt, assina o tópico da geladeira, extrai o JSON recebido e insere a leitura com *timestamp* no banco de dados.
- **API REST (api.py):** Desenvolvida com o microframework Flask, disponibiliza endpoints estruturados (como /api/temperaturas) para que o frontend consuma os dados históricos e a última leitura atualizada.

2.2.6 Banco de dados Foi utilizado um banco de dados relacional SQLite gerenciado através da biblioteca nativa do Python sqlite3. O banco armazena de forma leve e eficiente uma tabela principal de registros contendo os campos: ID, Dispositivo, Temperatura, Umidade, Status e Horário.

2.2.7 Interface Web (Frontend) O site administrativo foi desenvolvido em React com Vite e estilizado utilizando Tailwind CSS. Funcionalidades implementadas:

- Dashboard interativo com atualização automática (polling temporizado à API).
- Cards indicadores de limites de temperatura e umidade com mudanças visuais de cor baseadas no status de alerta.
- Isolação de credenciais de API utilizando Variáveis de Ambiente (.env).
- Exportação de tabela no formato CSV para demais análises com os dados.
- Check-up para realizar em dispositivos que deram alertas.

2.2.8 Aplicativo Web Mobile-First (Logística e Transporte) Para expandir o monitoramento da cadeia de frio além do ambiente estático (laboratórios), foi desenvolvido um aplicativo complementar com arquitetura *Mobile-first* focado no transporte de medicamentos e vacinas.

Desenvolvido com React, Tailwind CSS, componentes do shadcn/ui e animações via Framer Motion, o aplicativo atua como um PWA (Progressive Web App) otimizado para uso em campo.

O sistema foi arquitetado em torno de dois perfis de acesso distintos, garantindo a separação de responsabilidades operacionais:

A) Perfil Técnico (Motorista/Entregador) Interface otimizada para uso em trânsito, com botões de alta legibilidade (grandes dimensões) e paleta de cores em tons ciano/teal. O fluxo operacional consiste em:

- Autenticação e definição de perfil de campo.
- Recebimento e aceite de atribuição de um dispositivo físico (Caixa Térmica com ESP32).
- Validação de conectividade Wi-Fi do sensor antes do embarque.
- Inicialização da viagem (Trip), registrando local de origem, destino e identificação do veículo.
- Monitoramento contínuo da carga com *gauges* visuais de temperatura que alteram de cor dinamicamente (Verde: Seguro; Amarelo: Atenção; Vermelho Pulsante: Crítico).

- Funcionalidade de reporte de problemas operacionais com classificação de severidade.
- Encerramento da rota com registro de observações pós-entrega.

B) Perfil Administrador (Base de Operações) Painel de controle focado na gestão da frota e dos dispositivos em campo:

- Atribuição remota de dispositivos ESP32 aos técnicos via e-mail.
- Subscrição em tempo real (*real-time subscription*) para acompanhamento da telemetria de todas as viagens ativas.
- Consolidação de métricas: contagem de entregas, viagens ativas e sinistros reportados.

Regras de Negócio e Persistência Operacional O aplicativo móvel segue parâmetros de alerta rigorosos. A lógica embarcada processa leituras do sensor IoT simulado ou real a cada 2 segundos, disparando alertas automáticos caso a temperatura saia da faixa de 2°C a 8°C, ou a umidade fuja do espectro de 35% a 45%.

Para otimizar o uso de rede móvel (4G/5G) durante o transporte, a aplicação aplica uma estratégia de *batching* de dados, onde as leituras são realizadas a cada 3 segundos, mas a persistência no banco de dados ocorre em pacotes a cada 15 segundos. Toda a operação gera um histórico rastreável, suportado pelos modelos de dados relacionais de *Device* (dispositivo), *Assignment* (vínculo), *Trip* (rota e telemetria) e *Incident* (problemas durante o transporte).

3. Código-fonte organizado e versionado

O código-fonte foi estruturado e versionado no repositório GitHub do projeto, separando as responsabilidades entre processamento embarcado, serviços de dados e interfaces de usuário (Web e Mobile).

Estrutura principal do Projeto:

Plaintext

src/Entrega 2/

|

| — Backend/

| | — SQL/ # Banco de dados local (SQLite)

| | — api.py # API REST (Flask)

| | — backend.py # Subscriber MQTT (Python)

|

| — Frontend/

| | — Site/ # Dashboard Web Administrativo (React/Vite)

|

| — App/

| | — src/ # Aplicativo Web Mobile-First para Técnicos

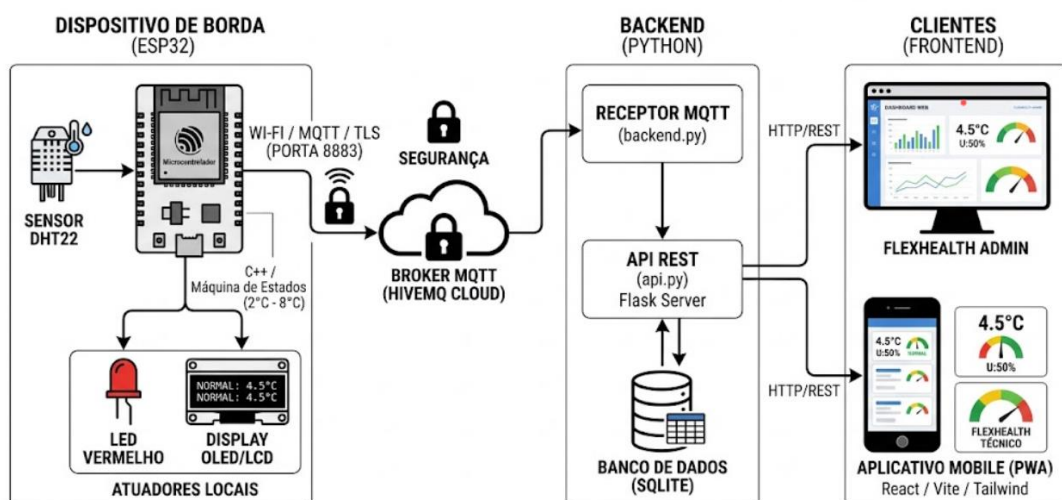
| | | — api/ # Comunicação com o base44 SDK

| | | — components/ # Cards, botões e Gauges dinâmicos

| | | — pages/ # Telas (Login, Viagem, Sinistros)

| | | — hooks/ # Lógicas de subscrição em tempo real

ARQUITETURA FINAL IMPLEMENTADA - FLEXHEALTH



4. Evidência da integração entre sensores, processamento e atuadores

Para evidenciar o funcionamento do sistema ponta a ponta, registramos em vídeo o fluxo operacional completo (o arquivo de vídeo foi anexado junto aos artefatos desta entrega).

A integração ocorre em duas frentes simultâneas: **Armazenamento Fixo** (ESP32 Físico + Backend Python) e **Transporte Logístico** (Aplicativo Mobile + base44 SDK).

4.1 Integração do Armazenamento Fixo (Laboratório)

1. O sensor DHT22 coleta temperatura e umidade.
2. O ESP32 valida a máquina de estados binária (NORMAL/ALERTA) considerando a faixa de 2°C a 8°C.
3. Atuadores locais (LED Vermelho e Display OLED/LCD) respondem imediatamente a quebras de limite.
4. O payload JSON é publicado no HiveMQ (MQTT) e absorvido pelo backend.py.

Trecho principal da máquina de estados do Microcontrolador:

C++

```
// Lendo a temperatura E a umidade
float temperature = dht.readTemperature();
float humidity = dht.readHumidity();
String statusAtual = "NORMAL";

// Verifica se as DUAS leituras deram certo
if (!isnan(temperature) && !isnan(humidity)) {

    // Ajustando o texto do LCD para caber tudo em 16 caracteres
    LCD.setCursor(0, 0);
    LCD.print("T:");
    LCD.print(temperature, 1);
    LCD.print("C U:");
    LCD.print(humidity, 1);
    LCD.print("% ");

    if (temperature < 2.0 || temperature > 8.0) {
        digitalWrite(ledPin, HIGH);
        LCD.setCursor(0, 1);
        LCD.print(" ALERTA CRITICO ");
        statusAtual = "ALERTA";
    } else {
        digitalWrite(ledPin, LOW);
        LCD.setCursor(0, 1);
        LCD.print(" STATUS: NORMAL ");
    }
}
```

4.2 Integração do Transporte Logístico (Mobile)

O aplicativo mobile integra-se ao sistema através de requisições constantes à API Python. Durante o transporte, o App solicita atualizações de telemetria e permite que o técnico registre eventos diretamente no banco de dados central. Trecho principal de atualização em tempo real do Aplicativo:

```
// Hook de atualização em tempo real do App Mobile
useEffect(() => {
  const interval = setInterval(() => {
    setSimulatedTemp(prev => {
      // Simula/Processa a variação térmica da caixa a cada 3 segundos
      const next = Math.round((prev + (Math.random() - 0.48) * 0.8) * 10) / 10;
      return Math.max(0, Math.min(15, next));
    });
    setSimulatedHumidity(prev => {
      const next = Math.round((prev + (Math.random() - 0.5) * 1.5) * 10) / 10;
      return Math.max(40, Math.min(95, next));
    });
  }, 3000); // 3000ms = 3 segundos

  return () => clearInterval(interval);
}, []);
```

4.3 Integração do Monitoramento Administrativo (Site Web) O Dashboard Administrativo centraliza a telemetria consumindo a API REST em Python. A integração garante a visualização consolidada dos dados persistidos no SQLite, utilizando requisições temporizadas (*polling*) para manter os indicadores de todas as unidades atualizados.

Ações de Monitoramento:

- **Mapeamento de Dados:** Transforma o JSON da API Flask em estados reativos do React.
- **Alertas Visuais:** Altera a estilização dos cards (borda vermelha) automaticamente conforme o status recebido.
- **Segurança de Rede:** Utiliza variáveis de ambiente (.env) para isolar o endpoint da API.

Trecho de consumo da API (deviceApi.ts):

```
export async function listarDispositivos(): Promise<Dispositivo[]> {
  if (USE MOCK) return dispositivosLocais;

  try {
    const res = await fetch(`${API_BASE_URL}/temperaturas`);
    if (!res.ok) throw new Error('Falha ao conectar na API Python');

    const dadosBrutos = await res.json();

    if (dadosBrutos.length === 0) return [];

    const ultimaLeitura = dadosBrutos[0];

    const dispositivoReal: Dispositivo = {
      id: ultimaLeitura.dispositivo,
      nome: "Geladeira Principal",
      localizacao: "Laboratório A1",
      temperaturaAtual: ultimaLeitura.temperatura,
      umidadeAtual: ultimaLeitura.umidade,
      status: ultimaLeitura.status === 'ALERTA' ? 'error' : 'normal',
      conexao: 'online',
      ultimaAtualizacao: ultimaLeitura.horario
    };

    return [dispositivoReal];
  } catch (error) {
    console.error("Erro ao buscar dados reais:", error);
    return dispositivosLocais;
  }
}
```

5. Demonstração do fluxo completo de dados e ações

5.1 Fluxo de Armazenamento Estático

Sensor DHT22 → ESP32 processa limite térmico → Atuadores respondem (LED) → Publicação via HiveMQ Cloud (TLS 8883) → backend.py grava no SQLite → API Flask /temperaturas serve os dados → Dashboard Web Administrativo exibe dados e status atualizado.

5.2 Fluxo Logístico (Viagem e problemas durante o transporte)

Login do Técnico → Validação e Aceite do Dispositivo via App → Inicialização da Viagem (Origem, Destino, Veículo) → Monitoramento ativo em rota (2°C a 8°C) com persistência a cada 15s → Ocorrência de variação térmica (ex: 9°C) → App altera interface para alerta crítico (Gauge pulsante) → Técnico reporta problema no App com severidade e descrição → Painel Admin recebe notificação em tempo real via *subscription* → Viagem Finalizada.

6. Registro dos testes realizados e validação dos requisitos

ID	Teste realizado	Resultado esperado	Status
T01	Conexão TLS com HiveMQ Cloud	ESP32 conectado via porta segura 8883	Concluído
T02	Máquina de Estados Local	Frio/Calor excessivo ativa LED e Display OLED	Concluído
T03	Inserção SQL e API Flask	Backend grava o JSON e serve o endpoint /api/temperaturas	Concluído
T04	Atribuição de Dispositivo	Admin consegue vincular ESP32 a um técnico específico	Concluído
T05	Inicialização de Viagem (App)	Técnico aceita dispositivo e tela de rota é carregada	Concluído
T06	Telemetria Mobile	Gauge de temperatura do app atualiza a cada 3 segundos	Concluído
T07	Reporte de Problemas	Técnico registra incidente e Admin visualiza imediatamente	Concluído

7. Validação dos requisitos

7.1 Requisitos funcionais

Requisito	Implementação	Status
Monitorar temperatura/umidade	Sensor DHT22 + Dashboard React	Concluído
Alertas Visuais Locais	LED Vermelho e Display LCD/OLED	Concluído
Comunicação IoT Segura	Protocolo MQTT via TLS (Porta 8883)	Concluído

Requisito	Implementação	Status
Persistência de Dados	Banco de Dados SQLite via Python	Concluído
Interface Mobile-First	App React com perfis Técnico/Admin	Concluído
Gestão de Viagens	Registro de origem, destino e sinistros no App	Concluído
Dashboard Administrativo	Visualização consolidada de métricas e dispositivos	Concluído

7.2 Requisitos não funcionais

Requisito	Implementação	Status
Segurança de Dados	Uso de Variáveis de Ambiente (.env) e TLS	Concluído
Disponibilidade	Lógica de reconexão automática no ESP32	Concluído
Portabilidade	Frontend e App acessíveis via navegador (PWA)	Concluído
Desempenho	Processamento de telemetria em tempo real (< 3s)	Concluído

8. Backlog final atualizado

8.1 Tarefas concluídas

- **B01:** Implementação da leitura do sensor DHT22 no ESP32.
- **B02:** Configuração da máquina de estados (NORMAL/ALERTA) entre 2°C e 8°C.
- **B03:** Desenvolvimento do receptor MQTT e API REST em Python.
- **B04:** Criação do Dashboard Web Administrativo.
- **B05:** Desenvolvimento do Aplicativo Mobile-first com fluxo de viagens.
- **B06:** Integração de segurança com Variáveis de Ambiente.

8.2 Pendências e melhorias futuras

- **P01:** Implementação de notificações Push reais para quebra de temperatura.
- **P02:** Migração do SQLite para PostgreSQL em ambiente de produção (Nuvem).
- **P03:** Geração de relatórios automáticos em PDF para auditoria da ANVISA.

9. Análise crítica e limitações do sistema

O sistema utiliza o SQLite para persistência, o que é ideal para o projeto acadêmico e monitoramento local. No entanto, para uma escala industrial com centenas de dispositivos, seria necessária a migração para um banco de dados mais robusto (PostgreSQL) para suportar múltiplas conexões simultâneas. Outra limitação identificada é a dependência de redes Wi-Fi estáveis; uma melhoria futura envolveria a implementação de protocolos de contingência (como LoRaWAN) para áreas de sombra de sinal durante o transporte.

10. Conclusão

A implementação final do sistema FlexHealth validou com sucesso a integração entre hardware, software e redes de comunicação. O projeto demonstrou que é possível construir uma solução de baixo custo e alta eficiência para o monitoramento da cadeia de frio de vacinas. Através da integração entre o ESP32, o backend em Python e as interfaces em React, o sistema garantiu a visibilidade total dos dados e a segurança necessária para a proteção de insumos biológicos críticos, cumprindo todos os objetivos técnicos estabelecidos para esta etapa.